

Helix Thready — Component Catalog

Table of Contents

1. How to read this catalog
2. Thready services (the deployable units)
 - 2.1 Per-component deep dive (responsibilities · interfaces · failure modes)
 - 2.1.1 Ingestion Service (extended Herald)
 - 2.1.2 Processing Service (Skill Dispatch Engine)
 - 2.1.3 Semantic-search Service
 - 2.1.4 Asset Service [BUILD-NEW]
 - 2.1.5 Download Manager [BUILD-NEW]
 - 2.1.6 User Service [BUILD-NEW]
 - 2.1.7 Event Bus Service [BUILD-NEW, thin]
 - 2.1.8 API Gateway
 - 2.1.9 Accounts (messenger) Service
3. Reused in-house submodules (the engines)
4. New submodules to build [BUILD-NEW]
5. Service → submodule dependency matrix
6. Component dependency diagram
7. Maturity & gap register
8. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (System Architecture)	Initial catalog — services, submodule mapping, maturity, gaps
2	2026-07-22	swarm (Pass 3 depth)	Add §2.1 per-component deep dive (responsibilities · interfaces · failure modes); fold verified <code>rag / cache / messaging / skill_registry / mcp_module</code> layouts (CAT-1/ CAT-2 closed); deepen matrix & dependency-diagram explanations
3	2026-07-22	swarm (Pass 3 depth, cont.)	Source-verify & correct import paths at source (<code>go.mod</code> read this pass): <code>SkillRegistry</code> module is <code>dev.helix.agent/skillregistry</code> (flat package — was mislabeled <code>digital.vasic.skill_registry</code>); MCP module is <code>digital.vasic.mcp</code> (was mislabeled <code>digital.vasic.mcp_module</code>); confirm <code>rag / cache / messaging pkg/</code> layouts at source and mark §3 rows VERIFIED; split §6 component-deps explanation into true multi-paragraph form; formally close CAT-1/CAT-2 in §8

Table of Contents

1. How to read this catalog
2. Thready services (the deployable units) 2.1. Per-component deep dive (responsibilities · interfaces · failure modes)
3. Reused in-house submodules (the engines)
4. New submodules to build `[BUILD-NEW]`
5. Service → submodule dependency matrix
6. Component dependency diagram
7. Maturity & gap register

8. Open items

1. How to read this catalog

Two kinds of components exist. **Services** are Thready's own deployable containers (they own an API surface and/or an event-loop). **Submodules** are decoupled, project-not-aware Go (or Python/Shell) libraries reused across Helix projects `[CONSTITUTION §11.4.28]`; Thready *composes* them, never forks them. Every service names the exact submodule(s) it embeds. Maturity is quoted verbatim from the gap register (`PRODUCTION` / `FOUNDATION` / `SCAFFOLD` / `DESIGN-ONLY` / `BUILD-NEW`) and confidence (`VERIFIED` / `FLAGGED`); a maturity claim is never upgraded here. Import paths use the confirmed `digital.vasic.*` module convention.

2. Thready services (the deployable units)

Service	Responsibility	Embeds (submodules)	Detailed doc
Ingestion Service (extended Herald)	Connect to messengers, read threads, assemble root+organic reply, persist, emit <code>post.received</code>	<code>herald</code> , <code>gotd/td</code> , Max adapter <code>[BUILD-NEW]</code> , <code>ThreadReader [BUILD-NEW]</code> , <code>database</code> , <code>eventbus</code>	messenger-ingestion.md
Processing Service (Skill Dispatch Engine)	Classify by hashtag/type, order & run Skills, post status reply	<code>helix_skills</code> , <code>background</code> , <code>eventbus</code> , <code>HelixLLM</code> , <code>llmprovider</code> , <code>HelixAgent</code> , <code>visionengine</code> +OCR	processing-pipeline.md
Semantic-search Service	Embed + index posts and generated materials; serve <code>/v1/search</code>	<code>embeddings</code> , <code>vectordb</code> (pgvector), <code>rag</code> , <code>HelixLLM</code> <code>/v1/embeddings</code> , <code>MCP_Module</code>	semantic-search.md
Asset Service <code>[BUILD-NEW]</code>	Store/secure/serve physical & virtual assets; <code>...-web</code> renditions; Range/HLS/DASH	<code>Catalogizer</code> , <code>storage</code> , <code>filesystem</code> , <code>security</code> , <code>auth</code>	asset-and-download.md
Download Manager <code>[BUILD-NEW]</code>	Multi-protocol byte fetch (HTTP/2/3, FTP/SMB/NFS/WebDAV); queue/resume/segment/progress/callback	<code>filesystem</code> , <code>http3</code> , <code>background</code>	asset-and-download.md
User Service <code>[BUILD-NEW]</code>	Multi-tenant users/roles/permissions (three-tier), accounts, memberships, billing/metering	<code>auth</code> , <code>security/pkg/policy</code> , <code>Catalogizer</code> RBAC pattern, <code>database</code>	security-model.md
Event Bus Service <code>[BUILD-NEW, thin]</code>	Client-facing subscription surface over JetStream; sticky/one-time; durable replay	<code>eventbus</code> (<code>pkg/nats</code>), <code>streaming</code> (WS hub)	event-model.md
API Gateway	HTTP/3 edge; <code>/v1</code> REST + WebSocket/SSE; auth/ratelimit/headers/CORS	<code>http3</code> , <code>middleware</code> , <code>auth</code> , <code>ratelimiter</code> , <code>security/pkg/headers</code>	system-overview.md
Accounts (messenger) Service	Interactive & non-interactive sign-in to messengers; session storage	<code>auth</code> , <code>security/pkg/securestorage</code> , <code>herald</code>	messenger-ingestion.md

2.1 Per-component deep dive (responsibilities · interfaces · failure modes)

The table above is the index; this section is the contract. For each deployable service it states the **responsibility boundary** (what it owns and, as importantly, what it does *not*), the **key interface** it exposes and the verified engine interface it consumes, and the **failure modes + degradation** behaviour that keeps

the aggressive SLO honest under partial failure. Interfaces marked VERIFIED were read at source this pass; the sibling doc carries the full design.

2.1.1 Ingestion Service (extended Herald)

- **Responsibility.** Own the messenger boundary: connect with operator accounts, stream + poll chosen channels, assemble the **root + organic reply** composite via the ThreadReader, run the verified self-filter so it never re-ingests its own status replies, persist the Complete Post to PostgreSQL, and emit `post.received`. It does **not** classify or process (that is Processing) and does **not** store bytes (that is the Asset Service) — it emits references and the event.
- **Interfaces.** Consumes the VERIFIED herald `commons_messaging/channels.Channel` (`Subscribe` / `SendReplyGeneric` / `BotSelfIdentity` / `DownloadAttachment`) and self-filter (`IsSelfEcho` / `StampSender`); publishes via `eventbus/pkg/nats.Publish`. Exposes the internal ThreadReader seam (`ListChannels` / `Poll` / `ReadThread` / `Reply` , `[BUILD-NEW]`). See `messenger-ingestion.md`.
- **Failure modes.** Telegram `FLOOD_WAIT` / rate-limit → circuit breaker backs off and honors the wait, channel marked `channel.health` degraded (sticky). Session expiry/auth loss → Accounts Service re-auth; ingestion for that account pauses, does not crash the process. MTPROTO history backfill is `[GAP: 5.1.1]` (QA-harness today); Max is `[GAP: 5.1.2]` (empty stub) — ingestion for Max is **explicitly incomplete**, never bluffed. Duplicate discovery (poll + push) is safe: dedup is downstream on `post_id`.

2.1.2 Processing Service (Skill Dispatch Engine)

- **Responsibility.** Own the post lifecycle after claim: classify (hashtags + indirect determination), resolve the Skill set from the HelixSkills DAG, order by the fixed precedence (`download > convert > analyze > research > reply`), run each step idempotently through the BackgroundTasks queue, embed artifacts, and post the status reply. It owns *orchestration*, not *knowledge* (`helix_skills`) or *byte fetch* (Download layer).
- **Interfaces.** Consumes `background.TaskQueue` / `TaskRepository` (VERIFIED — the `FOR UPDATE SKIP LOCKED` claim), `helix_skills` Skill-Graph, the LLM stack (HelixLLM/LLMProvider/HelixAgent), VisionEngine+OCR, and the Semantic-search service. `SkillRegistry = dev.helix.agent/skillregistry` (VERIFIED at source Pass 3 via `go.mod`: a **flat package** at the repo root — `executor.go`, `loader.go`, `manager.go`, `registry.go`, `storage.go` + `storage_memory.go` / `storage_postgres.go`, `validator.go`, `types.go`; **not** the earlier-quoted `digital.vasic.skill_registry` — import path corrected); tool exposure = `digital.vasic.mcp` (VERIFIED at source Pass 3 via `go.mod`: `pkg/{server,client,protocol,registry,adapter,config}`; the module path is `digital.vasic.mcp`, **not** `digital.vasic.mcp_module`). See `processing-pipeline.md`.
- **Failure modes.** Per-step failure → per-step retry/back-off (checkpointed via `TaskExecutor.Pause/Resume`), not whole-post rerun. Worker death → `StuckDetector` reclaim. LLM local outage → `LLMProvider` cloud fallback (breaker). Whole-post exhaustion → `dead_letter` + `post.failed`.

Unclassifiable post → `GenericIngest`, never dropped. Long research step must not hold a download slot — download is delegated to a separate pool (§2.1.5).

2.1.3 Semantic-search Service

- **Responsibility.** Own meaning-based retrieval over posts **and** all generated materials: chunk, embed (same model both sides), index into pgvector, serve `/v1/search`, and assemble RAG answers with citations. It owns the vector index; the relational store remains the SoR it hydrates from.
- **Interfaces.** Consumes VERIFIED `vectordb/pkg/client.VectorStore` (`Upsert` / `Search` / `Delete` / `Get`) and `embeddings/pkg/provider.EmbeddingProvider` (`Embed` / `EmbedBatch` / `Dimensions` / `Name`), plus `rag` (`pkg/{chunker,retriever,reranker,hybrid, pipeline}`) — VERIFIED layout). See `semantic-search.md`.
- **Failure modes. HashEmbedder trap** `[GAP: 2.1]` → startup guard fails loud rather than index garbage. Dimension mismatch → refuse. pgvector connection error → serve from cache / degrade to relational filter. Non-pgvector backends `[GAP: 3.1]` present-but-unhardened → config stays on pgvector for MVP. Cross-account leakage prevented by the `SearchQuery.Filter` account scope.

2.1.4 Asset Service `[BUILD-NEW]`

- **Responsibility.** Store, secure and serve physical **and** virtual assets: object tier (MinIO/S3), `...-web` renditions, HLS/DASH, Range serving, content-hash dedup, checksums, post↔asset links, per-account retention, and the sealed-directory decrypt path for sensitive scans. It owns storage/serving; it does **not** fetch bytes (Download layer) — they arrive via the standardized callback.
- **Interfaces.** Built on `Catalogizer` + `storage` + `filesystem` (`OpenSeekable` for Range) + `security` + `auth`; serves `GET /v1/assets/{id}`, `.../manifest.m3u8`, `POST .../redownload` (OpenAPI in `asset-and-download.md`); emits `asset.stored`.
- **Failure modes.** Broken physical link → `redownload` re-enqueues a delegated job. Transcode failure → raw preserved, `...-web` retried; serving falls back to raw. Sensitive decrypt requires an RBAC pass — a denied caller learns only that an asset exists. `[GAP: 6.1]` (Catalogizer not decoupled; `Streaming` =WS hub) means media byte-streaming/transcode is **new**, not existing.

2.1.5 Download Manager `[BUILD-NEW]`

- **Responsibility.** Generic multi-protocol byte fetch (HTTP/1.1/2/3+QUIC+Brotli, FTP/SMB/NFS/ WebDAV) with queue, resumable + segmented transfer, progress, retry/back-off, and a standardized completion callback. It fetches; it does not store/serve.
- **Interfaces.** `Enqueue` / `Status` / `Cancel` (`DownloadJob`); **reuses** `filesystem` for non-HTTP protocols and **adds** the HTTP source via `http3`; runs its **own worker pool** (does not consume the processing pool). Emits the shared callback → Asset Service.
- **Failure modes.** `[GAP: 6.2]` filesystem has no HTTP source + NFS mislisted on non-Linux → the new HTTP source and NFS platform-guard are part of the build. Source unavailable → retrievable callback →

requeue with back-off; exhausted → `post.failed`. Long transfers isolated by the dedicated pool so they cannot starve processing.

2.1.6 User Service [BUILD-NEW]

- **Responsibility.** Own tenancy: users, three-tier roles/permissions, accounts, memberships (many-to-many), branding/policy, and billing/metering. It is the home of the RBAC policy set.
- **Interfaces.** `auth` (JWT/API-key/OAuth2) + VERIFIED `security/pkg/policy.Enforcer` (`EvaluateAll` most-restrictive) + Catalogizer RBAC pattern + `database`. See `security-model.md`.
- **Failure modes.** JWT default HMAC-SHA256 is single-secret `[GAP: 7.2]` → RS256/EdDSA+JWKS planned before multi-service verification is trusted. Policy store empty → `EvaluateAll` defaults to Allow (so Thready always loads a deny-by-default cross-account policy at boot). Mobile secret storage stub `[GAP: 7.3]` → mobile clients get short-lived tokens only until native stores land.

2.1.7 Event Bus Service [BUILD-NEW, thin]

- **Responsibility.** Client-facing subscription surface over JetStream: sticky-snapshot-first delivery, durable per-client replay, WebSocket/SSE fan-out, RBAC-scoped subject filtering. Thin by design — the durability lives in JetStream.
- **Interfaces.** Wraps `eventbus/pkg/nats` (VERIFIED) **but drives the raw `JetStreamContext` directly** for durable consumers, because `pkg/nats.Subscribe` is ephemeral (`DeliverNew`) — see the anti-bluff boundary in `event-model.md` §1. Serves `WS /v1/events`, `SSE /v1/events/stream`.
- **Failure modes.** Client disconnect → durable consumer resumes from ack floor on reconnect. Outage beyond retention window → client reconciles via REST snapshots (`updated_since`). Sticky producer crash mid-invalidate → 24 h TTL backstop clears the stale value.

2.1.8 API Gateway

- **Responsibility.** Terminate client traffic (HTTP/3 QUIC + HTTP/2 fallback), apply auth, rate-limit, security headers and CORS, and expose the versioned `/v1` REST + realtime surfaces. It routes; business logic lives behind it.
- **Interfaces.** `http3` + `middleware` + `auth` + `ratelimiter` + `security/pkg/headers`. Fronts every service; never touches the DB directly except through service calls.
- **Failure modes.** Downstream service unhealthy → removed from the pool by the readiness gate (§ service-discovery). Rate-limit breach → 429 with token-bucket headers. TLS cert renewal handled by `lets_encrypt` atomic deploy-hook + rollback.

2.1.9 Accounts (messenger) Service

- **Responsibility.** Interactive and non-interactive sign-in to each messenger and encrypted session/token storage, keyed per account+messenger. Reusable/decoupled so any project adopts it.

- **Interfaces.** `auth` + VERIFIED `security/pkg/securestorage.Storage` (AES-256-GCM `FileStorage`; `IsSecure()` gates startup) + herald. See [messenger-ingestion.md](#) §7.
- **Failure modes.** Insecure store (`IsSecure()==false`) → **refuse to start**, never plaintext-fallback. 2FA/login-code flow is interactive; missing non-interactive creds → SKIP (never log, never crash). Session revocation propagates via the token store.

3. Reused in-house submodules (the engines)

Submodule (import path)	Repo	Role in Thready	Maturity (gap register)
<code>digital.vasic.database</code>	<code>vasic</code> <code>-digital/database</code>	Relational SoR — SQLite dev / Postgres prod; <code>pkg/migration.Runner</code>	PRODUCTION / VERIFIED
<code>digital.vasic.vectordb</code>	<code>vasic</code> <code>-digital/Vectordb</code>	Semantic store — pgvector backend, cosine <code><=></code>	PRODUCTION (pgvector) / VERIFIED — others FLAGGED [GAP: 3.1]
<code>digital.vasic.embeddings</code>	<code>vasic</code> <code>-digital/Embeddings</code>	Embedding generation (OpenAI-compat → HelixLLM)	PRODUCTION / VERIFIED — no native llama.cpp backend [GAP: 2.7]
<code>digital.vasic.rag</code>	<code>vasic</code> <code>-digital/RAG</code>	Retrieval-augmented generation glue; <code>pkg/{chunker,retriever,ranker,hybrid,pipeline}</code>	PRODUCTION / VERIFIED (layout re-read Pass 3)
<code>digital.vasic.eventbus</code>	<code>vasic</code> <code>-digital/EventBus</code>	In-proc bus (<code>pkg/bus</code>) + NATS JetStream (<code>pkg/nats</code>)	PRODUCTION / VERIFIED
<code>digital.vasic.background</code>	<code>vasic</code> <code>-digital/BackgroundTasks</code>	Postgres task queue, DLQ, retry/backoff, stuck-detect, Prometheus	PRODUCTION / VERIFIED
<code>digital.vasic.cache</code>	<code>vasic</code> <code>-digital/cache</code>	L1/L2 cache (in-mem + Redis + Postgres); <code>pkg/{memory,redis,postgres,distributed,policy,service}</code>	PRODUCTION / VERIFIED (layout re-read Pass 3)
<code>digital.vasic.storage</code>	<code>vasic</code> <code>-digital/Storage</code>	MinIO/S3 object tier + local FS + signed URLs	PRODUCTION / VERIFIED — MinIO signed-URL parity [GAP: 3.2]

Submodule (import path)	Repo	Role in Thready	Maturity (gap register)
<code>digital.vasic.filesystem</code>	<code>vasic-digital/Filesyste</code>	SMB/FTP/NFS/WebDAV/local; <code>OpenSeekable</code> for Range	PRODUCTION / VERIFIED — no HTTP(S) source [GAP: 6.2]
<code>vasic-digital/Catalogizer</code>	<code>vasic-digital/Catalogize</code>	Asset store/serve base (RBAC, WS, SQLCipher-at-rest)	PRODUCTION / VERIFIED — not decoupled; <code>Streaming</code> = WS hub [GAP: 6.1]
<code>herald</code>	<code>vasic-digital/herald</code>	Messenger fan-in/out; <code>pkg/messenger.Messenger</code>	FOUNDATION / VERIFIED — MTPROTO in QA harness, Max stub [GAP: 5.1]
<code>HelixDevelopment/HelixLLM</code>	<code>HelixDevelopment/HelixLLM</code>	Local llama.cpp serving; <code>/v1/embeddings</code> , <code>/v1/chat/*</code>	PRODUCTION / VERIFIED — default embedder is <code>HashEmbedder</code> stub [GAP: 2.1]
<code>digital.vasic.llmprovider</code>	<code>vasic-digital/LLMProvider</code>	40+ provider adapters; retry/circuit-breaker/health	PRODUCTION / FLAGGED (adapters not each audited)
<code>dev.helix.agent</code> (HelixAgent)	<code>HelixDevelopment/HelixAgent</code>	Ensemble/debate reasoning	FOUNDATION / FLAGGED — identity blur, residual stubs [GAP: 2.2]
<code>digital.vasic.llmsverifier</code>	<code>vasic-digital/LLMsVerifier</code>	Model scoring for fallback chain	PRODUCTION core / VERIFIED — port <code>:7061</code> vs <code>:8080</code> [GAP: 2.5]

Submodule (import path)	Repo	Role in Thready	Maturity (gap register)
<code>digital.vasic.visionengine</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>visio</code> <code>nengi</code> <code>ne</code>	LLM-vision adapters	FOUNDATION / VERIFIED — no OCR engine [GAP: 2.6]
<code>HelixDevelopment/</code> <code>helix_skills</code>	<code>Helix</code> <code>Devel</code> <code>opmen</code> <code>t/</code> <code>helix</code> <code>_skil</code> <code>ls</code>	Skill-Graph DAG (atomic→composite→umbrella)	FOUNDATION/MVP / VERIFIED — no execution engine [GAP: 4.1]
<code>digital.vasic.auth</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>auth</code>	JWT + API-keys + OAuth2	PRODUCTION / VERIFIED — JWT — default HMAC-SHA256 [GAP: 7.2]
<code>digital.vasic.security</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>secur</code> <code>ity</code>	AES-256-GCM + Argon2id; <code>pkg/</code> <code>securestorage</code> , <code>pkg/pii</code> , <code>pkg/policy</code>	PRODUCTION / VERIFIED
<code>digital.vasic.observability</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>obser</code> <code>vabil</code> <code>ity</code>	OTel + Prometheus + logrus + ClickHouse; <code>pkg/health</code>	PRODUCTION / VERIFIED
<code>digital.vasic.discovery</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>disco</code> <code>very</code>	Service discovery/scan; <code>pkg/</code> <code>report</code> , <code>pkg/scanner</code>	PRODUCTION / VERIFIED
<code>digital.vasic.mdns</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>mdns</code>	mDNS advertisement/browse	PRODUCTION / VERIFIED
<code>port_prefix</code>	<code>vasic</code> <code>-digi</code> <code>tal/</code> <code>port_</code>	Deterministic host-port bands (<code>Exposed(prefix,port,taken)</code>)	PRODUCTION / VERIFIED

Submodule (import path)	Repo	Role in Thready	Maturity (gap register)
	prefi x		
vasic-digital/http3	vasic- digi- tal/ http 3	quic-go/http3 transport wrapper	PRODUCTION / VERIFIED
digital.vasic.ratelimiter	vasic- digi- tal/ ratel imite r	Token-bucket / sliding-window limiting	PRODUCTION / VERIFIED
digital.vasic.middleware	vasic- digi- tal/ middl ewar e	CORS / request-id / recovery	PRODUCTION / VERIFIED
digital.vasic.streaming	vasic- digi- tal/ Strea ming	WebSocket hub (not media byte streaming)	PRODUCTION / VERIFIED [GAP: 6.1 note]
digital.vasic.messaging	vasic- digi- tal/ Messa ging	Kafka/RabbitMQ for firehose streams; pkg/ {broker,producer,consumer,ka fka,rabbitmq}	PRODUCTION / VERIFIED (layout re-read Pass 3)
milos85vasic/Boba-Base	milos 85vas ic/ Boba- Base	Torrent search/download; SSE + POST /api/v1/hooks	FOUNDATION / VERIFIED — bespoke callback [GAP: 6.4]
milos85vasic/YT-DLP (MeTube)	milos 85vas ic/ YT- DLP	Video/streaming download	FOUNDATION / VERIFIED — poll-only, no webhook [GAP: 6.5]
vasic-digital/lets_encrypt	vasic- digi- tal/	ACME certs (HTTP-01/DNS-01)	PRODUCTION / VERIFIED

Submodule (import path)	Repo	Role in Thready	Maturity (gap register)
	lets_encypt		
vasic-digital/containers	vasic-digital/containers	Rootless Podman orchestration	PRODUCTION [CONSTITUTION \$11.4.76]

4. New submodules to build [BUILD-NEW]

Each new submodule is decoupled (own repo under `vasic-digital / HelixDevelopment`, own `upstreams/` recipes, project-not-aware). Priority is from the gap register \$11.

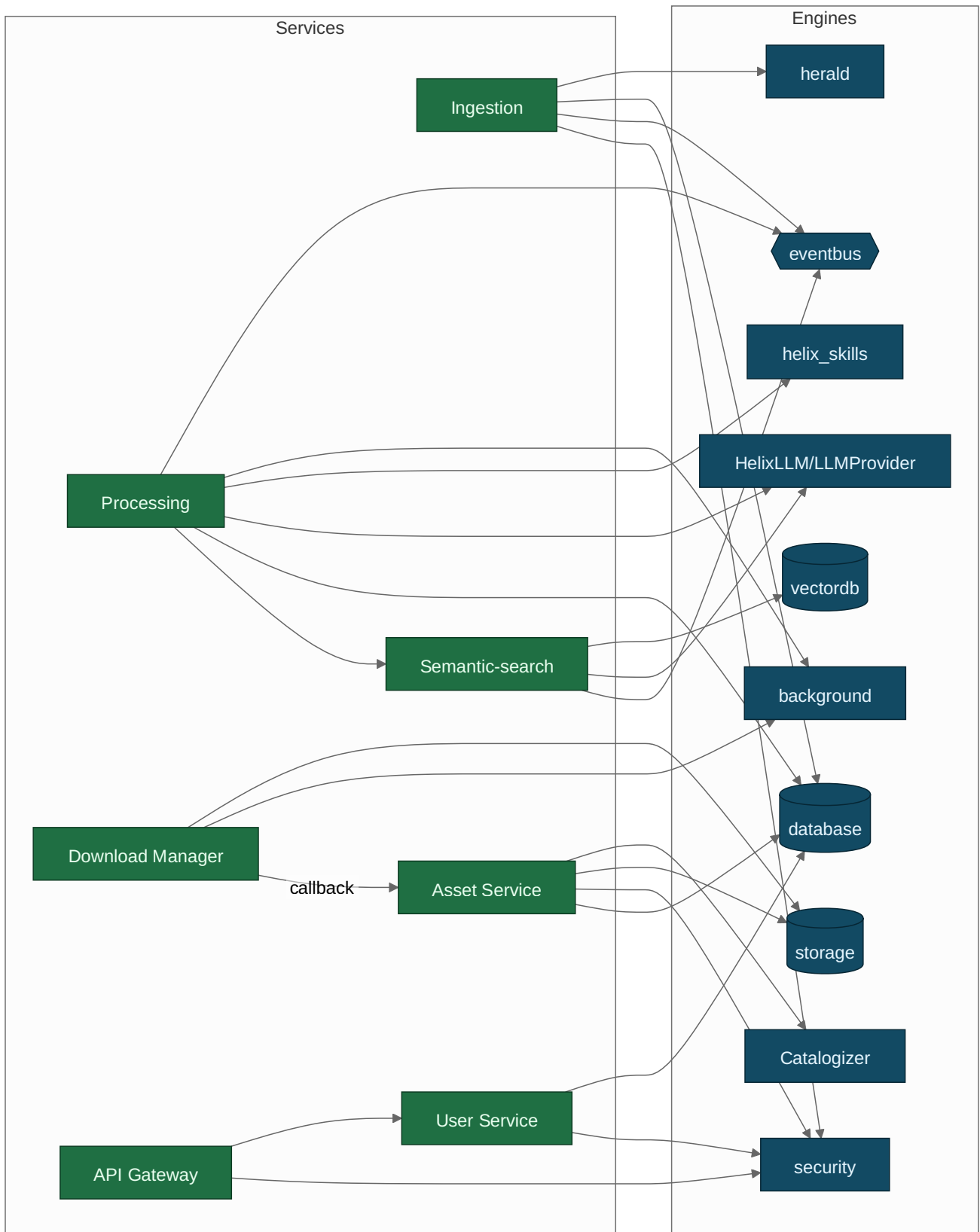
#	Submodule	Built on	Priority	Why new
1	Asset Service	Catalogizer + storage	P1	Decouple Catalogizer into a reusable asset store/serve
2	Download Manager	filesystem + http3	P0	No generic multi-protocol downloader exists [GAP: 6.3]
3	Max messenger adapter	herald <code>Messenger</code> seam	P0	Herald <code>max.go</code> is an empty stub [GAP: 5.1.2]
4	OCR adapter	visionengine seam	P0	VisionEngine has no OCR engine [GAP: 2.6]
5	User Service	auth + security/pkg/policy	P0	Three-tier multi-tenant RBAC service
6	MeTube completion webhook	MeTube sidecar	P0	Poll-only today [GAP: 6.5]
7	Standardized callback/task module	Boba/MeTube/DLM	P1	Uniform 3rd-party async contract [GAP: 6.6]
8	Event Bus service (thin)	eventbus (JetStream)	P1	Client-facing subscription surface
9	ThreadReader abstraction	herald channels	P1	Root+organic-reply assembly across messengers [GAP: 5.1.3]
10	Semantic-search service	embeddings+vectordb+rag	P0	Lumen-style in-house search service

5. Service → submodule dependency matrix

Service ↓ / Engine →	dat ab ase	vec tor db	eve ntb us	bac kgr ou nd	sto rag e	file sys tem	aut h	sec urit y	her ald	Hel ixL LM	hel ix_ skil ls	Cat alo giz er
Ingestion	•		•					•	•			
Processing	•	•	•	•				•	•	•	•	
Semantic-search	•	•	•							•		
Asset Service	•		•		•	•	•	•				•
Download Manager			•	•	•	•						
User Service	•		•				•	•				•
API Gateway							•	•				

Explanation. The matrix shows which engine each service embeds. Note three structural facts it encodes: (1) `security` is used by every data-touching service (encryption is not optional); (2) only the Processing and Semantic-search services depend on the LLM stack, which isolates the aggressive-SLO API path from slow model calls; (3) `eventbus` is nearly universal because the system is event-driven end-to-end — even the Asset Service emits `asset.stored`.

6. Component dependency diagram



Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: `diagrams/component-deps.mmd`.

Explanation (for readers/models that cannot see the diagram). The diagram is drawn as two columns — Thready services on the left, reused engines on the right — for a single structural reason the reader should hold onto: every edge points from a service to an engine and never the other way. That one-directional shape is what keeps the engines project-not-aware and reusable across Helix projects `[CONSTITUTION §11.4.28]`; the moment an engine depended on a Thready service it would stop being a general-purpose submodule. Edges mean “depends on / embeds”.

Reading the services in turn shows how narrow each dependency fan-out is. The Ingestion service depends on herald (messenger I/O), database (persistence), eventbus (emits `post.received`) and security (session encryption) — and notably *not* on the LLM stack, because ingestion never reasons, it only reads, assembles and persists. The Processing service is the busiest — it depends on eventbus and background (claim/queue), helix_skills (recipe knowledge), the LLM stack (research/analysis), database (state), and calls the Semantic-search service to index results. Semantic-search depends on vectordb (pgvector), the LLM stack (embeddings), and eventbus (emits `index.updated`).

The data-and-asset services form the last cluster, and it is where the one-way rule is easiest to misread, so the diagram is explicit about it. The Asset Service depends on Catalogizer + storage + security + database. The Download Manager depends on storage + background and, crucially, calls *back* into the Asset Service on completion (the dashed callback edge) — download and storage are separate concerns joined by the callback contract. The User Service and API Gateway sit on auth + security. No engine depends on a service (dependencies point one way, services → engines), which is what keeps the engines reusable in other Helix projects.

7. Maturity & gap register

Every component that the gap register flags as less-than-production is listed with its plan. None of these is claimed to “work” today; each has a `[GAP: ...]` and an owning design doc.

Component	Status	Gap headline	Plan (where)
HelixLLM embedder	PRODUCTION w/ trap	Default <code>HashEmbedder</code> is non-semantic	Enforce <code>HELIX_EMBEDDING_PROVIDER=llama</code> , fail loudly <code>[GAP: 2.1]</code> → semantic-search.md
VisionEngine	FOUNDATION	No OCR engine	Add Tesseract/PaddleOCR adapter <code>[GAP: 2.6]</code> → processing-pipeline.md
herald	FOUNDATION	MTPROTO in QA harness; Max empty	Promote MTPROTO reader; build Max adapter <code>[GAP: 5.1]</code> → messenger-ingestion.md
helix_skills	FOUNDATION	Knowledge units, no run engine	Build Skill Dispatch Engine <code>[GAP: 4.1]</code> → processing-pipeline.md
filesystem + Download Mgr	PRODUCTION / BUILD-NEW	No HTTP source; no download semantics	New Download Manager <code>[GAP: 6.2/6.3]</code> → asset-and-download.md
MeTube	FOUNDATION	Poll-only, no webhook	Add outbound webhook <code>[GAP: 6.5]</code> → asset-and-download.md
Catalogizer	PRODUCTION	Not decoupled; Streaming=WS hub	Decouple Asset Service <code>[GAP: 6.1]</code> → asset-and-download.md
auth	PRODUCTION	JWT default HMAC-SHA256	Add RS256/EdDSA + JWKS <code>[GAP: 7.2]</code> → security-model.md
Security-KMP	SCAFFOLD (mobile)	In-memory secret stub	Native Keystore/Keychain <code>[GAP: 7.3]</code> → security-model.md
vectordb (non-pgvector)	PRODUCTION (pgvector)	Qdrant/Pinecone/Milvus unverified	Harden Qdrant to parity <code>[GAP: 3.1]</code> → semantic-search.md
session_orchestrator	DESIGN-ONLY	Claim registry unimplemented	Thready single-claim reuses concept <code>[GAP: 2.9]</code> → concurrency-and-idempotency.md
database partitioning	PRODUCTION	No partition/shard helpers	Time-partition posts <code>[GAP: 3.2]</code> → data-flow.md

8. Open items

- `[CLOSED: CAT-1]` (was: `rag / messaging / cache` package layouts not re-read at source). **Source-verified this pass** (`gh api .../contents/pkg`): `digital.vasic.rag = pkg/{chunker,retriever,reranker,hybrid,pipeline}`; `digital.vasic.messaging` (repo `vasic-`

`digital/Messaging`) = `pkg/{broker,producer,consumer,kafka,rabbitmq}` (+ `i18n`);
`digital.vasic.cache` = `pkg/{cache,memory,redis,postgres,distributed,policy,service}`
 — i.e. the L1(in-mem)/L2(Redis)/Postgres tiers the matrix claimed. \$3 rows now marked VERIFIED. No residual verification needed for the layouts (adapter-level end-to-end hardening remains the gap register's general anti-bluff sweep, not a CAT open item).

- **[CLOSED: CAT-2]** (was: `MCP_Module` / `SkillRegistry` import paths FLAGGED docs-only). **Source-verified this pass** (`go.mod` read): the tool-exposure module is `digital.vasic.mcp` (repo `vasic-digital/MCP_Module`, `pkg/{server,client,protocol,registry,adapter,config}`) — **not** `digital.vasic.mcp_module`; the skill-registry module is `dev.helix.agent/skillregistry` (repo `vasic-digital/SkillRegistry`, a **flat package**: `executor.go` / `loader.go` / `manager.go` / `registry.go` / `storage.go` + `storage_postgres.go` / `validator.go` / `types.go`) — **not** `digital.vasic.skill_registry`. \$2.1.2 now carries the corrected paths. This is an anti-bluff correction: the previously-quoted `digital.vasic.*` paths were wrong and are fixed at source.

Made with love ♥ by Helix Development.